SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Comparative Analysis (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.	Bloom's Level: 4 – Analyze
2	Differentiate between clustered indexing and non-clustered indexing with suitable database examples.	Bloom's Level: 4 – Analyze
3	Compare primary indexing and secondary indexing based on storage requirements and search performance.	Bloom's Level: 4 – Analyze
4	Analyze the differences between static hashing and dynamic hashing in database systems.	Bloom's Level: 4 – Analyze
5	Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.	Bloom's Level: 4 – Analyze
6	Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.	Bloom's Level: 4 – Analyze
7	Compare dense indexing and sparse indexing with respect to memory usage and access speed.	Bloom's Level: 4 – Analyze

8	Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.	Bloom's Level: 4 – Analyze
9	Compare single-level indexing and multi-level indexing for efficient database retrieval operations.	Bloom's Level: 4 – Analyze
10	Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.	Bloom's Level: 5 – Evaluate

---

Code of Conduct:

I **VENNA MEENAKSHI REDDY(192471048)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Venna Meenakshi Reddy

RUBRICS FOR CALCULATION:

Criteria	Weigh tage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Compari son Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidenc e	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation
Clarity of Presenta tion	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Compare sequential file organization and heap file organization in terms of data insertion, deletion, and retrieval efficiency.

Sequential File Organization

Records are stored in a specific sequential order, usually based on a key field such as Student ID.

Heap File Organization

Records are stored without any particular order. A new record is generally placed wherever free space is available, often at the end of the file.

Aspect	**Sequential File Organization**	**Heap File Organization**
Data insertion	Slower because records may need to be placed in the correct sorted position	Very fast because the new record can be added to available space
Deletion	Relatively slower; deleting records may create gaps or require reorganization	Fast; record can simply be marked/deleted
Sequential retrieval	Very efficient because records are already ordered	Less efficient because records have no particular order
Exact-match retrieval	Efficient if searching using the ordering key, especially with binary search	Usually requires linear search unless an index exists
Range queries	Highly efficient because related records are stored close together	Inefficient because matching records may be scattered
Storage management	May require periodic reorganization	Easy to manage
Best suited for	Reports, payroll, transaction processing, range-based queries	Frequently changing data and simple record storage

Example

Suppose a student table is stored using Student_ID.

Sequential:

101 → Arun

102 → Bala

103 → Charan

104 → Divya

105 → Esha

Heap:

103 → Charan

101 → Arun

105 → Esha

102 → Bala

104 → Divya

If we want students with IDs 102–104, sequential organization can access them efficiently because they are stored together.

Critical Analysis

Sequential organization provides better retrieval and range-query performance, but insertion and deletion can become expensive. Heap organization provides excellent insertion and deletion performance, but retrieval is generally slower.

Conclusion:

Use sequential organization when ordered and range-based retrieval is important. Use heap organization when records are frequently inserted or deleted and ordering is not important.

2. Differentiate between clustered indexing and non-clustered indexing with suitable database examples.

A clustered index determines the physical order of records in the data file, whereas a non-clustered index is a separate structure that points to records stored elsewhere.

Aspect	**Clustered Index**	**Non-Clustered Index**
Physical ordering	Data records are physically arranged according to the index key	Data records are not physically arranged according to the index
Number per table	Usually one clustered ordering	Multiple non-clustered indexes can exist
Range queries	Very efficient	Less efficient than clustered for large ranges
Exact search	Efficient	Efficient
Insertion	Can be slower because physical order may need maintenance	Generally easier
Storage	Index structure plus organized data	Requires separate index storage
Best use	Range queries and ordered retrieval	Searches on frequently queried non-ordering attributes

Example

Consider:

STUDENT(Student_ID, Name, Department, Marks)

If the table is physically arranged by Student_ID:

101 → Arun

102 → Bala

103 → Charan

104 → Divya

Student_ID can be used as a clustered index.

Now suppose we create an index on Department:

CSE → 101, 103

ECE → 102

BIO → 104

This can be a non-clustered index because the actual records are not physically arranged by department.

Critical Analysis

Clustered indexing is particularly powerful for:

```
SELECT * FROM Student
WHERE Student_ID BETWEEN 100 AND 200;
```

because related records are physically close.

A non-clustered index is useful for:

```
SELECT * FROM Student
WHERE Department = 'CSE';
```

because the index quickly identifies the matching records.

Conclusion:

A clustered index generally provides better performance for range retrieval, while non-clustered indexes provide flexible access through multiple search attributes.

3. Compare primary indexing and secondary indexing based on storage requirements and search performance.

A primary index is generally created on the ordering primary-key field of a sorted file. A secondary index is created on a non-ordering field to provide an additional access path.

Aspect	Primary Index	Secondary Index
Search field	Usually primary key/order field	Non-ordering attribute
File ordering	Data file is ordered according to the search key	Data file is not ordered according to the index
Storage requirement	Usually lower, especially when sparse	Usually higher because it is commonly dense
Number of indexes	Generally one primary ordering index	Multiple secondary indexes can be created
Search performance	Very efficient for primary-key searches	Efficient for searches on indexed attributes
Range queries	Very efficient	Can be useful but may require many record accesses
Maintenance	Must maintain ordering	Additional index must be updated when records change

Example

Suppose:

Student_ID	Name	Department
101	Arun	CSE
102	Bala	ECE
103	Charan	CSE
104	Divya	BIO

A primary index can be created on:

Student_ID

A secondary index can be created on:

Department

Storage Difference

A primary index can often be sparse:

101 → Block 1

105 → Block 2

110 → Block 3

Instead of storing an index entry for every record.

A secondary index is often dense:

CSE → 101, 103

ECE → 102

BIO → 104

Therefore, secondary indexing can require more storage.

Critical Analysis

If the application frequently searches using the primary key, primary indexing provides excellent performance with relatively low index storage.

However, users may also search using fields such as:

- Name
- Department
- City
- Email

Secondary indexes provide these additional access paths but increase storage requirements and update overhead.

Conclusion:

Primary indexing is generally more storage-efficient and closely associated with the physical ordering of the file, while secondary indexing provides flexible searching at the cost of additional storage and maintenance.

4. Analyze the differences between static hashing and dynamic hashing in database systems.

Hashing uses a hash function to determine the bucket where a record should be stored.

Static hashing has a fixed number of buckets when the file is created. Dynamic hashing allows the number of buckets to grow or shrink as the database changes. Examples include extendible hashing and linear hashing.

Aspect	**Static Hashing**	**Dynamic Hashing**
Number of buckets	Fixed	Can change
Scalability	Poor for rapidly growing files	Excellent
Overflow handling	Overflow chains may develop	Buckets can split
Performance as data grows	Can decrease	Generally remains stable
Implementation	Simpler	More complex
Storage management	Less flexible	More flexible
Best suited for	Stable-size datasets	Frequently growing databases

Example

Suppose static hashing has four buckets:

h(key) = key MOD 4

If many records map to Bucket 2:

Bucket 2 → R1 → R5 → R9 → R13 → ...

an overflow chain develops.

In dynamic hashing, the system can split buckets to accommodate additional records.

Critical Analysis

Static hashing is simple and works well when the database size is predictable. However, if the number of records increases significantly, overflow chains can cause additional disk accesses.

Dynamic hashing requires more management but adapts to database growth.

Conclusion:

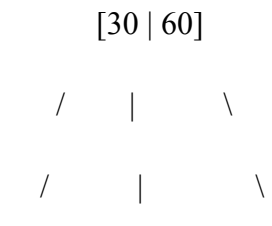
For a stable database, static hashing may be sufficient. For a large and continuously growing database, dynamic hashing provides better scalability and more consistent performance.

5. Compare B-Tree indexing and B+ Tree indexing for large-scale database applications.

Both B-Trees and B+ Trees are balanced multi-way search trees designed for efficient disk-based searching.

Aspect	**B-Tree**	**B+ Tree**
Data storage	Data/record pointers may occur in internal and leaf nodes	Actual record pointers are stored at leaf nodes
Internal nodes	Can contain keys and data pointers	Generally contain only keys and child pointers
Leaf nodes	May not necessarily be linked	Usually linked sequentially
Range queries	Less efficient	Highly efficient
Sequential access	Less convenient	Very efficient through linked leaves
Fan-out	Usually lower	Higher
Tree height	Can be relatively higher	Usually lower
Database usage	Possible, but less common	Widely preferred for database indexes

B+ Tree Example



[10 20] → [30 40 50] → [60 70 80]

The leaf nodes are linked:

[10,20] → [30,40,50] → [60,70,80]

This makes range searching very efficient.

For:

WHERE Marks BETWEEN 40 AND 70

the database can locate 40 and then scan the linked leaf nodes until 70.

Critical Analysis

B+ Trees have an important advantage in database systems because disk I/O is expensive. Their high fan-out reduces tree height, while linked leaves make sequential and range access efficient.

Conclusion:

Although both provide logarithmic search performance, B+ Trees are generally preferred for large-scale database applications, especially when range queries and sequential access are common.

6. Differentiate linear hashing and extendible hashing in terms of bucket management and scalability.

Both are dynamic hashing techniques, but they manage bucket growth differently.

Aspect	Linear Hashing	Extendible Hashing
Bucket splitting	Buckets split gradually in a predetermined sequence	Bucket splits occur based on overflow
Directory	No large directory is required	Uses a directory
Growth	Gradual	Can grow by doubling the directory
Memory requirement	Generally lower	Directory requires additional memory
Overflow handling	Controlled through progressive splitting	Bucket splitting reduces overflow
Scalability	Good	Excellent
Implementation	Relatively simpler	More complex
Best suited for	Gradual database growth	Applications requiring fast dynamic access

Linear Hashing

Suppose buckets are:

B0 B1 B2 B3

When splitting begins:

B0 → B0 + new B4

B1 → B1 + new B5

...

Splits occur progressively.

Extendible Hashing

Extendible hashing uses a directory:

00 → B0

01 → B1

10 → B2

11 → B3

When the directory needs more entries:

000 → ...

001 → ...

010 → ...

011 → ...

100 → ...

101 → ...

110 → ...

111 → ...

The directory can double in size.

Critical Analysis

Linear hashing avoids the memory overhead of a directory and expands gradually. Extendible hashing provides very fast bucket selection but requires directory management.

Conclusion:
Choose linear hashing when gradual expansion and lower directory overhead are important. Choose extendible hashing when fast dynamic access and efficient overflow handling are more important.

7. Compare dense indexing and sparse indexing with respect to memory usage and access speed.

A dense index contains an index entry for every search-key value or record, while a sparse index contains entries only for selected search-key values, usually one entry per data block.

Aspect	Dense Index	Sparse Index
Entries	Usually one entry per record/search value	Usually one entry per block
Storage	High	Low
Search speed	Very fast	Slightly slower
Memory usage	High	Low
File requirement	Can work with suitable indexing structures	Generally requires sorted/ordered data
Update overhead	Higher	Lower
Best suited for	Fast direct searching	Large files where storage efficiency matters

Example

Suppose a block contains:

Student_ID	Name
101	Arun
102	Bala
103	Charan
104	Divya

Dense Index

101 → Record 101

102 → Record 102

103 → Record 103

104 → Record 104

Sparse Index

101 → Block 1

If another block begins at 105:

101 → Block 1

105 → Block 2

Critical Analysis

Dense indexing provides faster direct access because every record has an index entry. However, the index itself can become very large.

Sparse indexing significantly reduces memory consumption but requires an additional search within the identified block.

Conclusion:

Dense indexing is preferred when fast access is more important than storage, while sparse indexing is better for large ordered files where index storage must be minimized.

8. Analyze the advantages and limitations of indexed sequential file organization versus direct file organization.

Indexed sequential file organization stores records sequentially according to a key, with an index used to locate blocks quickly. Direct file organization uses a hash function or direct-address mechanism to calculate the location of a record directly.

Aspect	**Indexed Sequential File Organization**	**Direct File Organization**
Storage order	Records are ordered by key	Records are placed according to calculated address
Sequential access	Excellent	Poorer
Exact-match search	Efficient	Extremely efficient with a good hash function
Range queries	Excellent	Poor
Insertion	May require overflow areas/reorganization	Usually fast
Deletion	Moderate complexity	Relatively simple
Hash collisions	Not applicable in the same way	Must be handled
Best suited for	Sequential and range processing	Exact-match retrieval

Example

Indexed sequential:

100 → 110 → 120 → 130 → 140 → 150

For:

Find records 120–140

the system can efficiently access a continuous section.

Direct organization:

h(120) → Bucket 3

h(130) → Bucket 7

It is excellent when we know the exact key.

Critical Analysis

Indexed sequential organization provides a good balance between sequential processing and indexed retrieval. Direct organization is superior for exact-match operations but performs poorly for range queries because records with nearby keys may be stored in completely different locations.

Conclusion:

Use indexed sequential organization for reports, ordered processing, and range searches. Use direct organization for fast exact-match access.

9. Compare single-level indexing and multi-level indexing for efficient database retrieval operations.

Single-level indexing has one index structure that directly points to data records or blocks. Multi-level indexing creates an index on another index, forming multiple levels.

Aspect	**Single-Level Index**	**Multi-Level Index**
Number of levels	One	Two or more
Search time	Can become slower for very large indexes	Faster for very large files
Storage	Simpler	Requires additional index levels
Memory requirement	Lower	Higher
Scalability	Limited for huge files	Excellent
Complexity	Simple	More complex
Disk access	More when index becomes large	Fewer accesses because each level narrows the search

Example

Suppose there are thousands of data blocks.

A single-level index might be:

Index

↓

Thousands of entries

↓

Data

Searching the large index itself may require several operations.

Multi-level indexing divides the index:

Top-Level Index

↓

Second-Level Index

↓

Data Blocks

Critical Analysis

Multi-level indexing is essentially an indexing hierarchy that reduces the amount of index that must be examined at each stage. This is particularly useful when the index itself becomes too large to efficiently search.

Conclusion:

Single-level indexing is suitable for small or moderately sized files, while multi-level indexing is more appropriate for large-scale databases where minimizing disk I/O is critical.

10. Evaluate the performance of hashing techniques and indexing techniques for exact-match and range queries in databases.

Hashing and indexing are optimized for different types of queries.

Query Type	**Hashing**	**Indexing**
Exact-match query	Excellent	Excellent
Range query	Poor	Excellent
Sequential access	Poor	Excellent with ordered indexes
Ordering	Not preserved	Can preserve ordering
Average exact lookup	Approximately O(1) with good hashing	Typically O(log n) for tree indexes
Range retrieval	Requires scanning many buckets/records	Efficient with B+ Tree
Collision issue	Yes	No hash collision issue
Best example	WHERE Student_ID = 105	WHERE Marks BETWEEN 70 AND 90

Exact-Match Query

Consider:

SELECT *

FROM Student

WHERE Student_ID = 105;

A hash index can calculate:

h(105) → Bucket

and directly access the required bucket.

Therefore, hashing is generally extremely efficient for exact-match searches.

Range Query

Consider:

SELECT *

FROM Student

WHERE Marks BETWEEN 70 AND 90;

Hashing is not suitable because:

70, 71, 72, ... 90

may be distributed across unrelated buckets.

A B+ Tree index maintains keys in sorted order:

60 → 65 → 70 → 75 → 80 → 85 → 90 → 95

Therefore, the database can locate 70 and scan until 90.

Performance Evaluation

For exact matching:

Hashing

↓

Calculate hash

↓

Locate bucket

↓

Retrieve record

For range queries:

B+ Tree

↓

Find starting key

↓

Follow linked leaf nodes

↓

Retrieve range

Critical Analysis

Hashing can outperform indexing for exact-match queries because it can provide near-constant average lookup time. However, hashing does not maintain the ordering of keys, making it unsuitable for range operations.

B+ Tree indexing has slightly higher search overhead for exact matches but is much more versatile because it supports:

- Exact-match queries
- Range queries
- Ordered retrieval
- Sequential traversal

Conclusion:

Hashing is generally the better choice for exact-match queries, while B+ Tree indexing is superior for range queries and ordered retrieval. Therefore, the appropriate technique should be selected according to the application's dominant query workload rather than assuming one method is universally better.